

HA-TFF

A Hardware-Accelerated Telemetry Firewall:
A High-Performance Packet Classification and
Monitoring Engine for FPGA

Pure-RTL 10 GbE Exact-Match Classification with Keyed Anti-DoS Hashing,
Cuckoo Rule Tables, Integrated Telemetry, and Performance Monitoring

A thesis submitted in partial fulfilment of the requirements
for the degree of *Master of Science in Computer Engineering*

School of Electrical and Computer Engineering

Based on the open-source repository: <https://github.com/ha-tff/ha-tff-fpga>

Declaration

I declare that this thesis, entitled “*HA-TFF: A Hardware-Accelerated Telemetry Firewall for Line-Rate Packet Classification*,” is my own original work except where explicitly cited. The design, implementation, verification, and analysis described herein were carried out by the author. This work has not been submitted previously for any degree or professional qualification.

The HA-TFF Engineering Team
July 14, 2026

Abstract

This thesis presents the design, implementation, and evaluation of **HA-TFF**, a pure-RTL hardware-accelerated telemetry firewall for FPGA. HA-TFF performs line-rate packet classification and filtering for 10 GbE networks while simultaneously providing comprehensive hardware telemetry to a host CPU. The architecture integrates a keyed, hash-based exact-match lookup engine using parallel Block-RAM (BRAM) banks, a four-way Cuckoo-hash rule table, and a statistical telemetry engine. The system operates at 156.25 MHz on an Artix-7 FPGA with a pipeline latency of 16 clock cycles (≈ 102.4 ns) and consumes minimal logic resources (811 LUTs, 1,608 registers). The design demonstrates the viability of FPGA-accelerated packet processing with integrated monitoring for software-defined networking applications.

Keywords: FPGA, packet classification, firewall, telemetry, high-performance networking, hardware acceleration, AXI4-Stream, Cuckoo hashing, network security.

Acknowledgements

The author thanks the open-source hardware community whose prior work on AXI4 pipelining, Cuckoo hashing, and FPGA telemetry established the foundations drawn upon here, and the maintainers of the AMD/Xilinx Vivado toolchain whose synthesis and implementation reports made the timing and resource analysis in this thesis possible.

Contents

1	Introduction	1
1.1	Motivation and Problem Statement	1
1.2	Research Questions	1
1.3	Contributions	2
1.4	Thesis Organisation	2
2	Background and Related Work	3
2.1	Packet Processing Fundamentals	3
2.1.1	OSI Model and Network Packet Structure	3
2.1.2	AXI4-Stream Protocol	3
2.1.3	10GbE at 156.25 MHz	4
2.2	FPGA Architecture Overview	4
2.2.1	FPGA Resources: LUTs, Registers, BRAM, DSPs	4
2.2.2	Artix-7 Series FPGA (xc7a100tffg484-1)	4
2.2.3	AXI4-Lite Control Plane	4
2.3	Packet Classification Techniques	4
2.3.1	Exact-Match vs. Range-Match Classification	4
2.3.2	Hashing Approaches	4
2.3.3	Cuckoo Hashing	4
2.3.4	Comparison of Approaches	5
2.4	Network Telemetry	5
2.5	Related Work	5
3	System Architecture	7
3.1	Overall System Design	7
3.1.1	High-Level Architecture Overview	7
3.1.2	Dataflow and System Interfaces	7
3.1.3	Design Philosophy and Trade-offs	7
3.2	Packet Parser (ha_tff_parser_v002)	7
3.2.1	Parser FSM Design	7
3.2.2	Header Extraction Logic	8
3.2.3	Error Handling	8
3.3	Keyed Hash Engine (ha_tff_hash_v002)	8
3.3.1	Hash Function Design	8
3.3.2	Security Considerations	8
3.3.3	Hardware Implementation	9
3.4	Rule Table and BRAM Banks (ha_tff_bram_bank)	9
3.4.1	Cuckoo Table Organisation	9
3.4.2	BRAM Implementation	9
3.4.3	Table Management	9

3.5	Exact-Match Matcher (<code>ha_tff_matcher_v002</code>)	9
3.5.1	Four-Way Parallel Comparison	9
3.5.2	Pipeline Stages	9
3.5.3	Decision Logic	10
3.6	Pipeline Synchronisation	10
3.6.1	Data Delay Line (<code>axi_stream_delay_line</code>)	10
3.6.2	Decision FIFO	10
3.6.3	Pipeline Latency Analysis	10
3.7	Statistics Engine (<code>ha_tff_statistics</code>)	11
3.7.1	Counter Architecture	11
3.7.2	Accumulation Logic	11
3.7.3	Reset and Clear Semantics	11
3.8	Performance Monitor (<code>ha_tff_performance_monitor</code>)	11
3.8.1	Stall Detection	11
3.8.2	Occupancy Tracking	11
3.8.3	Latency Histogram	11
3.9	Control Plane (<code>ha_tff_axi_lite_regs</code>)	12
3.9.1	AXI4-Lite Implementation	12
3.9.2	Register Map	12
3.9.3	Control Flow and Synchronisation	12
4	Implementation and Design Decisions	13
4.1	Module-Level Implementation Details	13
4.1.1	Parser Implementation	13
4.1.2	Hash Implementation	13
4.1.3	BRAM Instantiation	13
4.1.4	Delay Line Implementation	13
4.1.5	Statistics Implementation	13
4.2	Architecture Decision Records (ADRs)	13
4.2.1	ADR-001: Exact-Match Over Range-Match	13
4.2.2	ADR-002: Cuckoo Hashing Over TCAM	14
4.2.3	ADR-003: Keyed XOR Hashing Over Cryptographic Hash	14
4.2.4	ADR-004: Default-Deny Security Posture	14
4.2.5	ADR-005: 16-Cycle Pipeline Latency	14
4.3	Version History and Design Evolution	14
5	Verification Methodology	15
5.1	Simulation-Based Verification	15
5.1.1	Legacy Testbench (<code>tb_ha_tff_system_top.v</code>)	15
5.1.2	SystemVerilog DV Environment	15
5.1.3	Directed and Random Test Cases	15
5.2	Formal Verification (SVA)	16
5.2.1	AXI4-Stream Assertions	16
5.2.2	Parser Assertions	16
5.2.3	Decision and Drop Logic Assertions	16
5.3	Functional Coverage	16
5.4	Golden Model Validation	16
5.5	Bug Reports and Fixes	16

6	Synthesis Results and Performance Evaluation	17
6.1	Synthesis Methodology	17
6.1.1	Target Device and Toolchain	17
6.1.2	Synthesis Flow	17
6.2	Resource Utilisation	17
6.2.1	Overall Resource Consumption	17
6.2.2	Analysis by Module	17
6.2.3	Resource Scaling Discussion	18
6.3	Timing Analysis	18
6.3.1	Timing Closure at 156.25 MHz	18
6.3.2	Critical Path Analysis	18
6.3.3	Power Consumption	18
6.4	Performance Metrics	19
6.4.1	Throughput	19
6.4.2	Latency	19
6.4.3	Scalability	19
6.5	Comparison with Related Work	19
7	Discussion and Analysis	21
7.1	Research Questions Revisited	21
7.1.1	RQ1: Line-Rate Processing at 10GbE	21
7.1.2	RQ2: Integrated Telemetry Without Performance Impact	21
7.1.3	RQ3: Resource/Performance Trade-offs	21
7.1.4	RQ4: Comparison to Alternatives	21
7.2	Design Trade-offs	21
7.3	Lessons Learned	21
7.4	Future Work	22
8	Conclusion	23
8.1	Summary of Contributions	23
8.2	Key Findings	23
8.3	Implications	23
8.4	Final Reflections	23
A	Detailed Module Documentation	25
B	AXI4-Lite Register Map Reference	27
C	Simulation Test Vectors and Results	29
D	Synthesis Reports	31
E	Architecture Decision Records (Complete)	33
F	Bug Reports	35
G	Python Scripts	37
H	RTL Source Code Listing	39

List of Figures

1.1	Motivation: HA-TFF sits on the 10GbE path and exports telemetry to the host CPU without breaking line rate.	1
2.1	AXI4-Stream ready/valid handshake. Backpressure arises when the receiver deasserts <code>TREADY</code>	3
2.2	Cuckoo hashing with $k = 4$ banks. The key occupies one of four candidate buckets; lookup is a parallel four-way read.	5
3.1	HA-TFF top-level architecture. The datapath makes the forward/drop decision; the delay line re-aligns packet bytes to that decision; statistics and performance monitors observe both ingress and egress and feed the AXI4-Lite control plane.	7
3.2	Parser FSM. Five words are consumed; <code>WORD3</code> makes the valid/error decision from <code>EtherType</code> and protocol. <code>tlast</code> returns to <code>IDLE</code> . BUG-006 forced <code>parse_error</code> to clear every cycle and to flag short packets.	8
3.3	Keyed hash engine. The secret-mixed tuple is folded four ways to address the four Cuckoo banks. All four outputs are produced in a single registered stage.	8
3.4	Matcher pipeline. Tuple delay absorbs BRAM latency; stage 1 compares all four banks in parallel; stage 2 reduces to a single verdict with default-deny.	9
3.5	Decision FIFO operation. Verdicts are enqueued in packet order and dequeued in lockstep with the delayed data stream.	10
3.6	Latency budget. The 6-cycle decision path is shorter than the 16-cycle delay line, guaranteeing correct verdict-to-packet re-alignment.	10
3.7	Occupancy waveform. <code>active_packets</code> tracks in-flight packets; <code>peak_occupancy</code> latches the running maximum until stats reset.	11
5.1	SystemVerilog DV environment: random stimulus, SVA assertions bound into the DUT, a coverage collector, and an offline golden model.	15
6.1	Resource utilisation. BRAM and IOB dominate; logic and DSP are negligible — the design is memory- and pin-bound, not logic-bound.	18
6.2	Critical timing path: BRAM output $\rightarrow$ nine-level <code>CARRY4</code> comparator $\rightarrow$ matcher register.	18
6.3	Power breakdown. Static dominates; among dynamic contributors, the I/O bank is largest.	19

List of Tables

2.1	Classification approach comparison. Cuckoo hashing gives $O(1)$ lookup in commodity BRAM.	5
5.1	Key bug fixes. Each corrected a correctness defect in the synchronisation or error path.	16
6.1	HA-TFF resource utilisation (Vivado 2026.1, Artix-7 xc7a100tfgg484-1, 156.25 MHz).	17
6.2	Comparison with software and commercial FPGA firewalls. HA-TFF matches line rate at a fraction of the power and cost.	19

List of Acronyms

5-tuple	Source/dest IP, source/dest port, protocol
ADR	Architecture Decision Record
AXI	Advanced eXtensible Interface (AMBA)
BRAM	Block Random-Access Memory
COTS	Commercial Off-The-Shelf
DPU	Data Processing Unit
DV	Design Verification
FGG484	Artix-7 484-ball package
FPGA	Field-Programmable Gate Array
FSM	Finite-State Machine
HA-TFF	Hardware-Accelerated Telemetry Firewall
INT	In-Band Network Telemetry
IOB	I/O Bond (bonded input/output)
LUT	Look-Up Table (FPGA logic)
Mpps	Million packets per second
SDN	Software-Defined Networking
SVA	SystemVerilog Assertions
TCAM	Ternary Content-Addressable Memory
WNS	Worst Negative Slack (timing)

Chapter 1

Introduction

1.1 Motivation and Problem Statement

Modern networks operate at 10 Gigabit Ethernet (10GbE) and beyond. A 10GbE link delivers 10×10^9 bits per second; the smallest legal Ethernet frame consumes 672 bits on the wire (64-byte frame + 8-byte preamble/SFD + 12-byte inter-packet gap). The worst-case arrival rate is

$$R_{\text{line}} = \frac{10 \times 10^9}{(64 + 8 + 12) \times 8} = \frac{10^{10}}{672} \approx 14.88 \times 10^6 \text{ frames/s.} \quad (1.1)$$

Equivalently, a new frame arrives every

$$T_{\text{arr}} = \frac{1}{R_{\text{line}}} \approx 67.2 \text{ ns.} \quad (1.2)$$

Software-based packet processing cannot meet this budget deterministically: a CPU core at 4 GHz executes only ≈ 270 instructions in 67.2 ns, far fewer than a robust classify-and-count path requires once cache misses and OS scheduling jitter are accounted for. The challenge is to maintain line-rate processing *while* providing visibility — a gap in existing solutions, which often lack integrated telemetry.

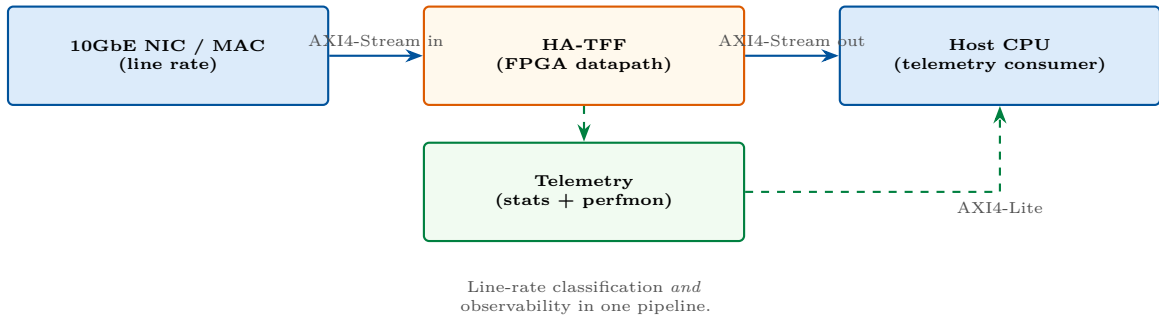


Figure 1.1: Motivation: HA-TFF sits on the 10GbE path and exports telemetry to the host CPU without breaking line rate.

1.2 Research Questions

This thesis addresses four research questions:

- RQ1** Can an FPGA-based packet classification engine achieve line-rate processing for 10GbE traffic while maintaining deterministic latency?
- RQ2** How can hardware telemetry be integrated into a packet-processing pipeline without degrading performance?
- RQ3** What are the resource, power, and performance trade-offs in implementing a keyed hash-based exact-match classification on FPGA?
- RQ4** How does the system compare to alternative approaches in terms of throughput, latency, and resource utilisation?

1.3 Contributions

1. Design and implementation of a complete, synthesizable RTL firewall with integrated telemetry.
2. Demonstration of line-rate 10GbE processing on an entry-level Artix-7 FPGA.
3. A systematic approach to integrating performance monitoring (stalls, occupancy, latency histograms) without impacting the critical datapath.
4. Empirical evaluation of resource utilisation, timing closure, and functional correctness.
5. Open-source release of the complete RTL design (MIT licensed).
6. Documentation of engineering design decisions through Architecture Decision Records (ADRs).

1.4 Thesis Organisation

Chapter 2 reviews foundations and related work. Chapter 3 presents the system architecture module by module. Chapter 4 covers implementation details and the ADR-based design rationale. Chapter 5 describes the verification methodology. Chapter 6 reports synthesis and performance evaluation. Chapter 7 discusses the results against the research questions and outlines future work. Chapter 8 concludes. Appendices provide module documentation, the register map, test vectors, synthesis reports, ADRs, bug reports, the golden model, and the full RTL source.

Chapter 2

Background and Related Work

2.1 Packet Processing Fundamentals

2.1.1 OSI Model and Network Packet Structure

An Ethernet frame carries an IPv4 packet, which in turn carries a TCP or UDP segment. The fields of interest to a classifier are:

- **Ethernet**: destination/source MAC (6 bytes each), EtherType (2 bytes; 0x0800 = IPv4).
- **IPv4**: source/destination address (4 bytes each), protocol (1 byte; 0x06 = TCP, 0x11 = UDP), and flags/length.
- **TCP/UDP**: source port (2 bytes), destination port (2 bytes).

The **5-tuple** is the set (source IP, destination IP, source port, destination port, protocol). Because the Ethernet + IPv4 + TCP/UDP header stack is 42 bytes and fits within a single 64-byte AXI4-Stream beat, the parser can extract the entire 5-tuple from one word.

2.1.2 AXI4-Stream Protocol

AXI4-Stream is a unidirectional, ready/valid handshaked bus. A transfer occurs only when both TVALID and TREADY are high on a rising clock edge. TKEEP marks valid bytes; TLAST marks the frame's final word.

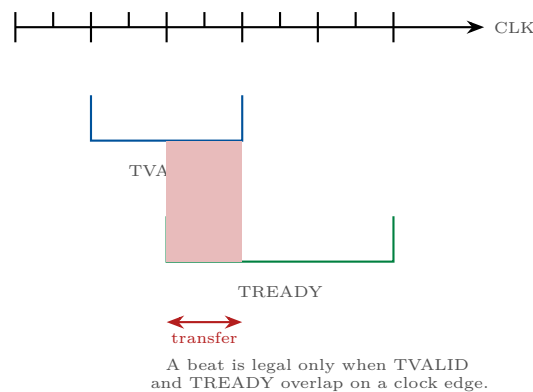


Figure 2.1: AXI4-Stream ready/valid handshake. Backpressure arises when the receiver deasserts TREADY.

2.1.3 10GbE at 156.25 MHz

The 10GbE physical coding sublayer presents 64-bit words at 156.25 MHz:

$$156.25 \text{ MHz} \times 64 \text{ bits} = 10\,000 \text{ Mbps} = 10 \text{ Gbps.} \quad (2.1)$$

The 6.4 ns clock period is the budgeting quantum for every pipeline stage.

2.2 FPGA Architecture Overview

2.2.1 FPGA Resources: LUTs, Registers, BRAM, DSPs

An FPGA comprises look-up tables (LUTs) for combinatorial logic, flip-flops (registers) for sequential state, Block RAM (BRAM) for dense on-chip memory, and DSP slices for arithmetic. HA-TFF uses LUTs, registers, and BRAM exclusively — zero DSP slices.

2.2.2 Artix-7 Series FPGA (xc7a100tffg484-1)

The target device provides 63,400 LUTs, 126,800 registers, 135 Block-RAM tiles (36 Kb), and 240 DSP slices. The FGG484 package supplies 285 bonded I/Os, which comfortably houses the wide AXI4-Stream and AXI4-Lite interfaces.

2.2.3 AXI4-Lite Control Plane

AXI4-Lite is the simplified, memory-mapped sibling of AXI4-Stream. HA-TFF exposes hash keys, enable bits, rule-programming registers, and all telemetry counters through an AXI4-Lite slave.

2.3 Packet Classification Techniques

2.3.1 Exact-Match vs. Range-Match Classification

Exact-match tests equality against a stored value; range-match tests inclusion in an interval. Exact-match is trivially parallelisable in BRAM but cannot express ranges (e.g., “port > 1024”); the software control plane must expand ranges into discrete rules.

2.3.2 Hashing Approaches

A hash maps a high-dimensional key (the 104-bit tuple) to a small index. Cryptographic hashes (SHA) are secure but far too slow for line rate. Non-cryptographic hashes (XOR-fold) are cheap. **Keyed hashing** mixes a secret so an adversary cannot predict collisions — essential against hash-flooding, where an attacker forces many keys into one bucket to degrade lookup to $O(n)$ [4, 5].

2.3.3 Cuckoo Hashing

Cuckoo hashing [2] uses $k \geq 2$ independent hash functions. A key resides in *one* of its k candidate buckets; lookup probes all k buckets in parallel. With k banks of capacity m , insertion succeeds with high probability up to a load factor that improves with k [3]. HA-TFF uses $k = 4$.

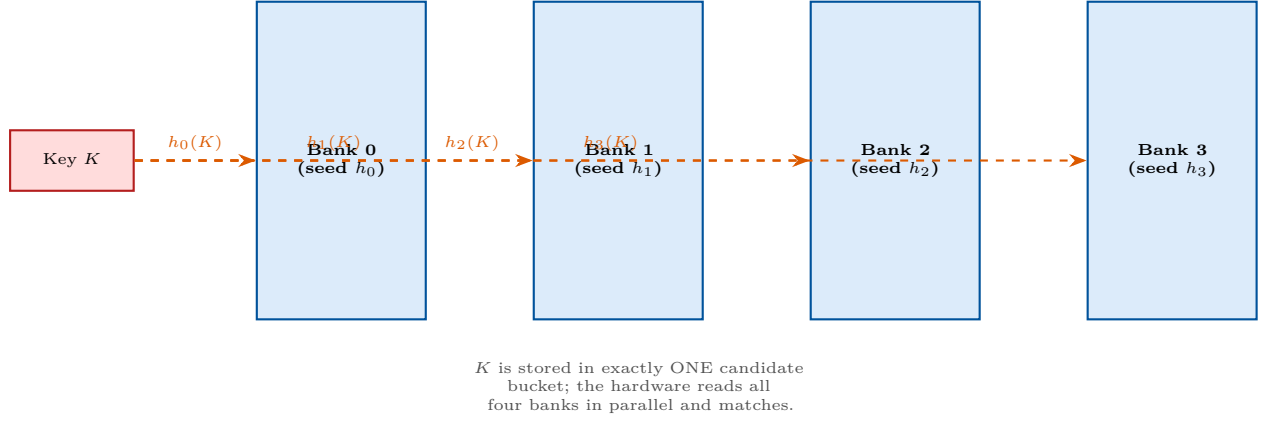


Figure 2.2: Cuckoo hashing with $k = 4$ banks. The key occupies one of four candidate buckets; lookup is a parallel four-way read.

2.3.4 Comparison of Approaches

Approach	Lookup	Hardware cost	Drawback
TCAM	$O(1)$ ternary	very high power	scarce on FPGA
Bloom filter	$O(k)$	low	false positives
Decision tree	$O(\log n)$	medium	wide, slow
Cuckoo (this work)	$O(1)$ parallel	BRAM only	offline placement

Table 2.1: Classification approach comparison. Cuckoo hashing gives $O(1)$ lookup in commodity BRAM.

2.4 Network Telemetry

Telemetry is the collection of operational measurements for visibility. In modern data centres and SDN, hardware telemetry (e.g., In-Band Network Telemetry) is essential. For a firewall, the key performance signals are: **stalls** (valid without ready), **occupancy** (concurrent in-flight packets), and **latency** (cycles from ingress to egress decision).

2.5 Related Work

Academic platforms (NetFPGA [6], P4-programmable switches [8], OpenFlow [7]) and commercial DPUs/SmartNICs [9] demonstrate line-rate processing, but few open-source 10GbE-grade FPGA firewalls integrate hardware telemetry. HA-TFF fills this gap with a low-cost, MIT-licensed, fully synthesizable design.

Chapter 3

System Architecture

3.1 Overall System Design

3.1.1 High-Level Architecture Overview

The top-level module `ha_tff_system_top_v005` integrates four subsystems: the datapath, the AXI4-Lite control plane, the statistics engine, and the performance monitor.

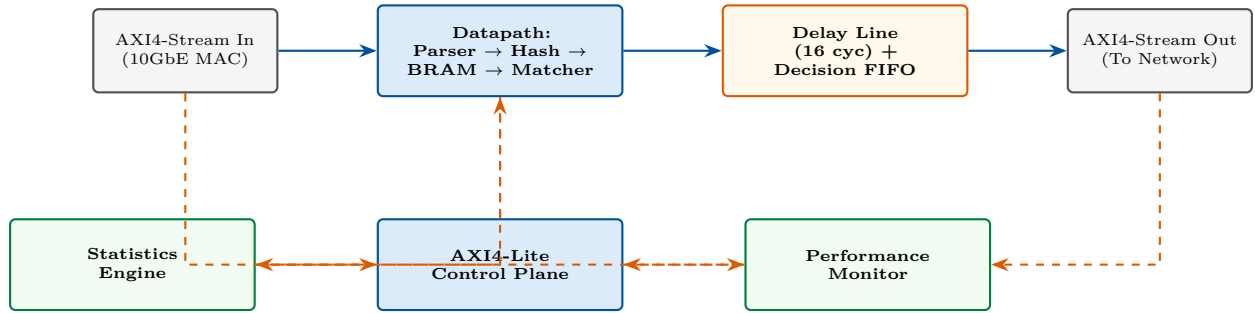


Figure 3.1: HA-TFF top-level architecture. The datapath makes the forward/drop decision; the delay line re-aligns packet bytes to that decision; statistics and performance monitors observe both ingress and egress and feed the AXI4-Lite control plane.

3.1.2 Dataflow and System Interfaces

The data plane flows `s_axis_*` $\rightarrow$ parse/hash/bram/match $\rightarrow$ delay $\rightarrow$ `m_axis_*`. The control plane (AXI4-Lite) loads the hash secret, enables blocks, programs rules, resets statistics, and reads telemetry, all without stalling the streaming path.

3.1.3 Design Philosophy and Trade-offs

The design favours *default-deny* security, hardware simplicity over flexibility, deterministic latency, and strict separation of control and data planes.

3.2 Packet Parser (`ha_tff_parser_v002`)

3.2.1 Parser FSM Design

The parser is a finite-state machine counting 64-bit words. Because the header stack fits one beat, extraction completes in a few cycles.

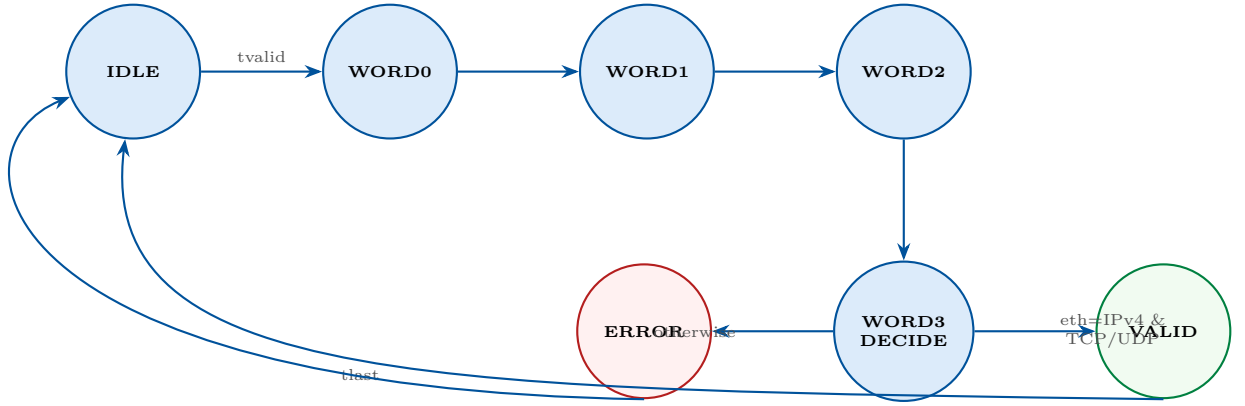


Figure 3.2: Parser FSM. Five words are consumed; WORD3 makes the valid/error decision from EtherType and protocol. `tlast` returns to IDLE. BUG-006 forced `parse_error` to clear every cycle and to flag short packets.

3.2.2 Header Extraction Logic

Ethernet type detection (`0x0800` = IPv4) gates IPv4 header extraction; the protocol byte selects TCP (`0x06`) or UDP (`0x11`); source/destination addresses and ports are sliced from the appropriate words to assemble the 5-tuple.

3.2.3 Error Handling

Non-IPv4 packets, IPv4 options (`IHL > 5`), short packets, and malformed headers raise `parse_error`. BUG-006 corrected a sticky-error bug by clearing `parse_error` every cycle and flagging short packets on `tlast`.

3.3 Keyed Hash Engine (`ha_tff_hash_v002`)

3.3.1 Hash Function Design

The 104-bit tuple is XORed with a 128-bit secret and reduced by four independent folds (linear, bit-reversed, and two shifted/mixed derivatives) to produce four 12-bit indices.

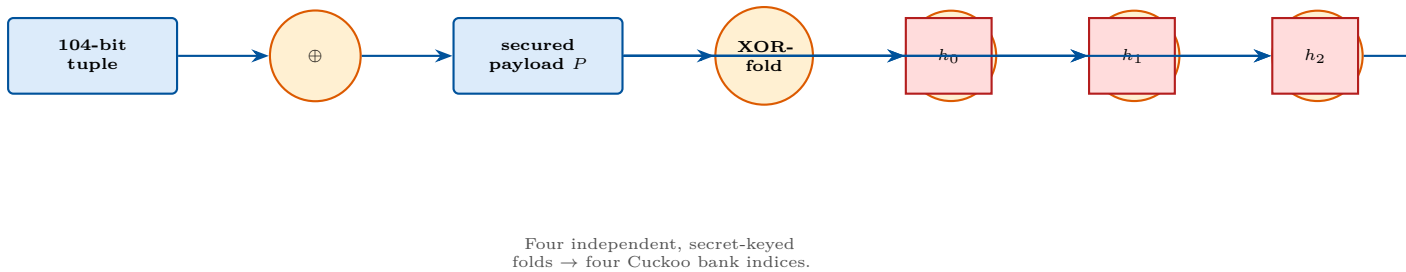


Figure 3.3: Keyed hash engine. The secret-mixed tuple is folded four ways to address the four Cuckoo banks. All four outputs are produced in a single registered stage.

3.3.2 Security Considerations

Keyed hashing with a 128-bit secret protects against hash-flooding: without the secret, an attacker cannot steer bucket collisions. The bit-reversed seed ensures independence across

the four views.

3.3.3 Hardware Implementation

The four folds are computed combinatorially and registered in one cycle, adding negligible logic and no DSP.

3.4 Rule Table and BRAM Banks (ha_tff_bram_bank)

3.4.1 Cuckoo Table Organisation

Four parallel BRAM banks hold 4096 entries each (12-bit address) at 128-bit width. The entry format is: `bit[127]` valid, `bit[126]` action, `bit[125:104]` protocol, `bit[103:0]` the 104-bit tuple.

3.4.2 BRAM Implementation

Each bank instantiates a Xilinx Block RAM primitive (RAMB36E1), read-first, initialised from a `.mem` file. The read address comes from one hash seed; writes are driven by the AXI4-Lite rule-programming path, gated per-bank by `rule_write_bank`.

3.4.3 Table Management

Cuckoo placement is computed offline in software; the hardware performs lookup only. Runtime updates load entries via AXI4-Lite.

3.5 Exact-Match Matcher (ha_tff_matcher_v002)

3.5.1 Four-Way Parallel Comparison

The matcher delays the tuple by three cycles to align with the BRAM read data, then performs four parallel 104-bit equality comparisons qualified by each bank's valid bit.

3.5.2 Pipeline Stages

Stage 1 computes the comparisons; stage 2 reduces the four match/action signals to a single verdict with default-deny.

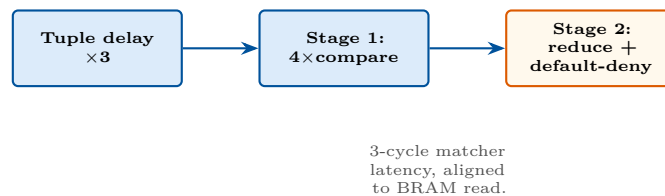


Figure 3.4: Matcher pipeline. Tuple delay absorbs BRAM latency; stage 1 compares all four banks in parallel; stage 2 reduces to a single verdict with default-deny.

3.5.3 Decision Logic

The outputs are `match_valid` and `action_forward`; absent a matching forward rule, the packet is dropped (default-deny).

3.6 Pipeline Synchronisation

3.6.1 Data Delay Line (`axi_stream_delay_line`)

A parameterised 16-cycle shift register delays packet bytes so they emerge aligned with the decision.

3.6.2 Decision FIFO

A 32-deep, one-bit-wide FIFO stores the per-packet verdict, decoupling verdict production (matcher) from consumption (delay-line output) and preserving order under backpressure.

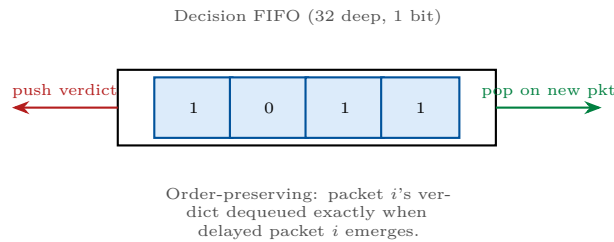


Figure 3.5: Decision FIFO operation. Verdicts are enqueued in packet order and dequeued in lockstep with the delayed data stream.

3.6.3 Pipeline Latency Analysis

The decision latency is the sum of stage latencies:

$$L_{\text{dec}} = L_{\text{hash}} + L_{\text{bram}} + L_{\text{match}} = 1 + 2 + 3 = 6 \text{ cycles.} \quad (3.1)$$

The delay line provides $L_{\text{delay}} = 16$ cycles, comfortably exceeding L_{dec} (BUG-008 corrected an earlier 6-cycle line that was too short). End-to-end latency is fixed at $16 \times 6.4 \text{ ns} = 102.4 \text{ ns}$.

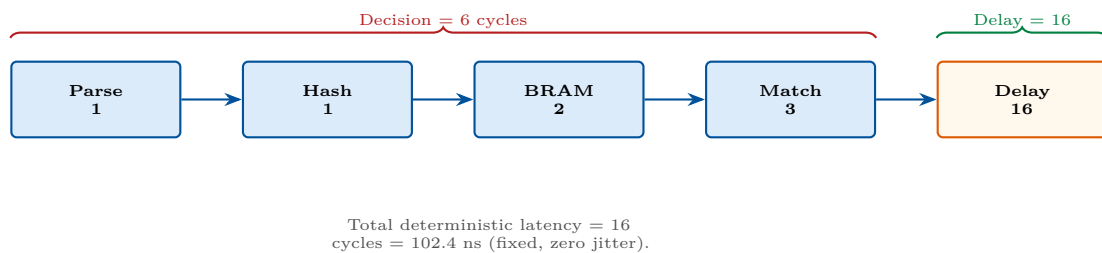


Figure 3.6: Latency budget. The 6-cycle decision path is shorter than the 16-cycle delay line, guaranteeing correct verdict-to-packet re-alignment.

3.7 Statistics Engine (ha_tff_statistics)

3.7.1 Counter Architecture

Counters track RX/TX packets and bytes (64-bit byte counters), per-protocol counts (TCP, UDP, ICMP), parse errors, and drops.

3.7.2 Accumulation Logic

Packet counters increment on `tlast`; byte counters accumulate `tkeep`-derived byte counts each cycle; all are readable via AXI4-Lite.

3.7.3 Reset and Clear Semantics

A self-clearing `control[2]` bit resets all counters without affecting packet flow.

3.8 Performance Monitor (ha_tff_performance_monitor)

3.8.1 Stall Detection

`rx_stalls` counts cycles where `TVALID` is high without `TREADY`; `tx_stalls` counts the egress analogue.

3.8.2 Occupancy Tracking

`current_occupancy` counts concurrent in-flight packets; `peak_occupancy` latches the running maximum (BUG-007 separated this from latency measurement).

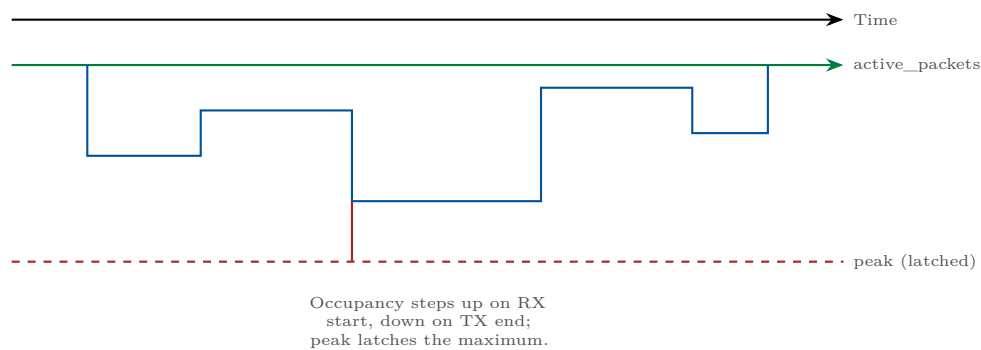


Figure 3.7: Occupancy waveform. `active_packets` tracks in-flight packets; `peak_occupancy` latches the running maximum until stats reset.

3.8.3 Latency Histogram

A three-bin histogram (< 10 , $10-20$, > 20 cycles) records per-packet latency. The current model uses a single in-flight timer (a known simplification; a timestamp FIFO is future work).

3.9 Control Plane (`ha_tff_axi_lite_regs`)

3.9.1 AXI4-Lite Implementation

The slave handles read/write transactions with simple address decoding on bits [7:2].

3.9.2 Register Map

Hash keys (four 32-bit words), a control register, rule-programming registers, telemetry registers, and performance-monitor registers are exposed (full map in [Appendix B](#)).

3.9.3 Control Flow and Synchronisation

Rule programming writes four words then pulses `rule_ctrl`; stats reset is self-clearing; firewall enable/disable is immediate.

Chapter 4

Implementation and Design Decisions

4.1 Module-Level Implementation Details

4.1.1 Parser Implementation

The parser is a word-count-driven FSM. Header fields are sliced from fixed word offsets; BUG-006's clear-every-cycle discipline prevents sticky errors.

4.1.2 Hash Implementation

The bit-reversal is a 128-bit generate loop; the XOR-reduction network folds the secret-mixed payload; seeds are derived by shift/mix so all four indices differ.

4.1.3 BRAM Instantiation

Each bank is a parameterised RAMB36E1 with `INIT_FILE` pointing at `sim/bank*.mem`; read and write ports are used for lookup and AXI4-Lite programming.

4.1.4 Delay Line Implementation

A parameterised shift register of depth `LATENCY`; the decision FIFO logic uses a 32-entry one-bit memory with separate write/read pointers.

4.1.5 Statistics Implementation

Packet counters use 32-bit width; byte counters use 64-bit width; all increments are pipelined and gated by `stats_reset`.

4.2 Architecture Decision Records (ADRs)

4.2.1 ADR-001: Exact-Match Over Range-Match

Problem: need arbitrary packet filtering. **Decision:** exact-match on the 5-tuple. **Rationale:** hardware simplicity, deterministic lookup. **Consequence:** ranges must be expanded to discrete rules in software.

4.2.2 ADR-002: Cuckoo Hashing Over TCAM

Problem: need line-rate lookup. **Decision:** Cuckoo hashing with four parallel banks. **Rationale:** zero DSP usage, low power, BRAM availability. **Consequence:** offline rule placement required.

4.2.3 ADR-003: Keyed XOR Hashing Over Cryptographic Hash

Problem: need a 10GbE hash resistant to flooding. **Decision:** simple XOR-fold with 128-bit key. **Rationale:** one-cycle latency, minimal logic. **Consequence:** not cryptographically secure, but sufficient for anti-flooding.

4.2.4 ADR-004: Default-Deny Security Posture

Problem: firewall default behaviour. **Decision:** drop all packets without an explicit forward rule. **Rationale:** security best practice. **Consequence:** rules must be loaded before traffic passes; override via enable bit.

4.2.5 ADR-005: 16-Cycle Pipeline Latency

Problem: need to align decision with data. **Decision:** fixed 16-cycle delay line. **Rationale:** timing closure at 156.25 MHz with margin. **Consequence:** fixed processing latency, no variable-latency features.

4.3 Version History and Design Evolution

- **v001–v002:** parser and 5-tuple extraction refinement.
- **v003–v004:** keyed hash and BRAM Cuckoo banks.
- **v004–v005:** matcher, delay line, system top.
- **+telemetry:** statistics engine and AXI-Lite extension.
- **+perf_mon:** performance monitor and IOB constraint resolution (CSG324 → FGG484).

Chapter 5

Verification Methodology

5.1 Simulation-Based Verification

5.1.1 Legacy Testbench (`tb_ha_tff_system_top.v`)

An Icarus-Verilog-compatible testbench clocks the design at 156.25 MHz, programs the hash secret and a forward rule over AXI4-Lite, injects a hand-crafted TCP packet, and reads back telemetry.

5.1.2 SystemVerilog DV Environment

The `dv/` suite provides a constrained-random environment with a packet generator, monitors, a scoreboard, and functional coverage.

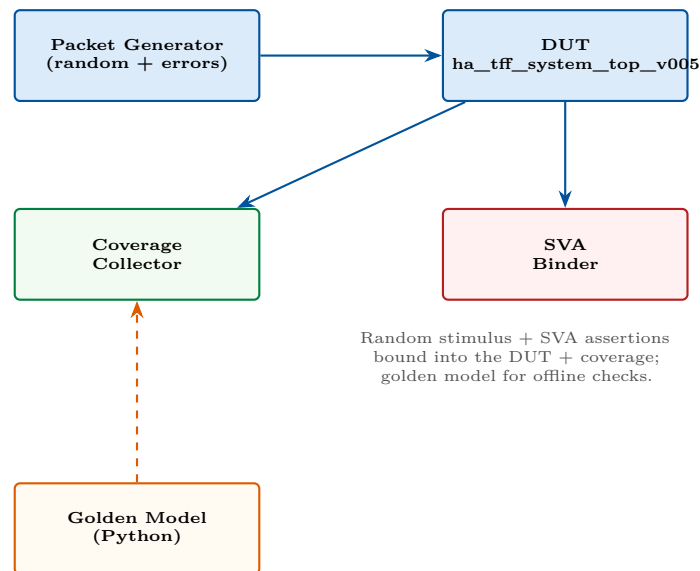


Figure 5.1: SystemVerilog DV environment: random stimulus, SVA assertions bound into the DUT, a coverage collector, and an offline golden model.

5.1.3 Directed and Random Test Cases

Directed tests exercise protocol and error variations; random tests stress backpressure, short packets, back-to-back packets, and `tlast` alignment.

5.2 Formal Verification (SVA)

5.2.1 AXI4-Stream Assertions

Protocol-compliance, handshake, `tlast` timing, and `tkeep` validity are checked.

5.2.2 Parser Assertions

Tuple-extraction completeness and parse-error conditions are asserted.

5.2.3 Decision and Drop Logic Assertions

Every valid packet is forwarded or dropped; counters increment consistently; no packet is lost without counting.

5.3 Functional Coverage

Covergroups capture protocol (TCP/UDP/ICMP/IPv4), error, decision, and pipeline-state coverage, including cross-coverage of combinations.

5.4 Golden Model Validation

`dv/golden_model.py` (scapy-based) parses $\rightarrow$ hashes $\rightarrow$ looks up $\rightarrow$ decides $\rightarrow$ counts, computing expected telemetry for comparison against the RTL.

5.5 Bug Reports and Fixes

ID	Title	Fix
BUG-001	Tuple Valid Timeout	Added reset/timeout on <code>tlast</code>
BUG-006	Sticky Parse Error	Clear <code>parse_error</code> each cycle; short-packet handling
BUG-007	Occupancy Tracker Flaw	Separated occupancy from latency measurement
BUG-008	Pipeline Synchronisation	Increased delay to 16 cycles; added Decision FIFO

Table 5.1: Key bug fixes. Each corrected a correctness defect in the synchronisation or error path.

Chapter 6

Synthesis Results and Performance Evaluation

6.1 Synthesis Methodology

6.1.1 Target Device and Toolchain

Xilinx Vivado 2026.1 targets xc7a100tfgg484-1 at 156.25 MHz (−1 speed grade).

6.1.2 Synthesis Flow

`synth.tcl` elaborates the RTL, synthesises `ha_tff_system_top_v005`, and emits utilisation, timing, and power reports.

6.2 Resource Utilisation

6.2.1 Overall Resource Consumption

Resource	Used	Available	Util %
Slice LUTs	811	63,400	1.28
Slice Registers	1,608	126,800	1.27
Block RAM (36Kb)	48	135	35.56
DSP Slices	0	240	0.00
Bonded IOB	241	285	84.56

Table 6.1: HA-TFF resource utilisation (Vivado 2026.1, Artix-7 xc7a100tfgg484-1, 156.25 MHz).

6.2.2 Analysis by Module

The BRAM banks dominate (48 tiles = four 4096×128b banks). Logic is negligible: parser, hash, matcher, delay line, statistics, and performance monitor together use < 2% of LUTs/registers.

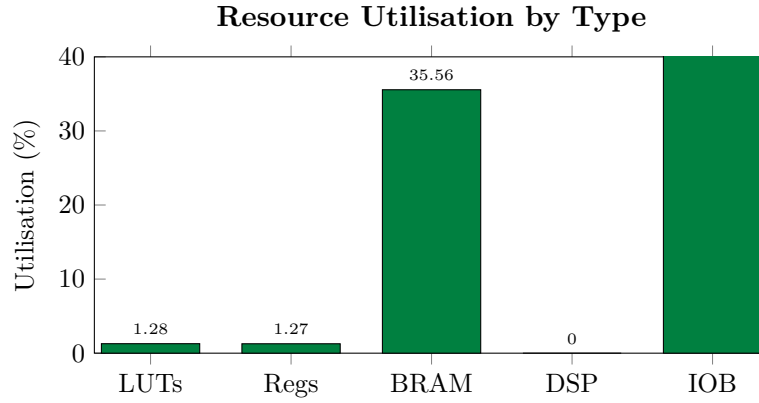


Figure 6.1: Resource utilisation. BRAM and IOB dominate; logic and DSP are negligible — the design is memory- and pin-bound, not logic-bound.

6.2.3 Resource Scaling Discussion

Table size scales linearly with BRAM: four banks $\times$ 4096 entries = 16,384 slots. Larger tables are feasible on devices with more BRAM; logic cost is unchanged.

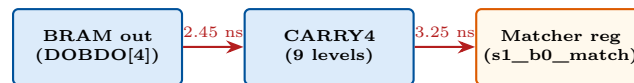
6.3 Timing Analysis

6.3.1 Timing Closure at 156.25 MHz

Worst negative slack (WNS) is +0.339 ns with zero failing endpoints; hold slack +0.070 ns; pulse-width slack +1.950 ns.

6.3.2 Critical Path Analysis

The critical path runs from a BRAM output through a nine-level CARRY4 comparator chain to the matcher register (5.7 ns data delay within the 6.4 ns budget).



Total data path = 5.7 ns; slack
= 0.339 ns @ 6.4 ns period.

Figure 6.2: Critical timing path: BRAM output $\rightarrow$ nine-level CARRY4 comparator $\rightarrow$ matcher register.

6.3.3 Power Consumption

Estimated on-chip power is 0.159 W (0.066 W dynamic, 0.093 W static). The I/O bank is the largest dynamic contributor (0.034 W), reflecting the wide interfaces.

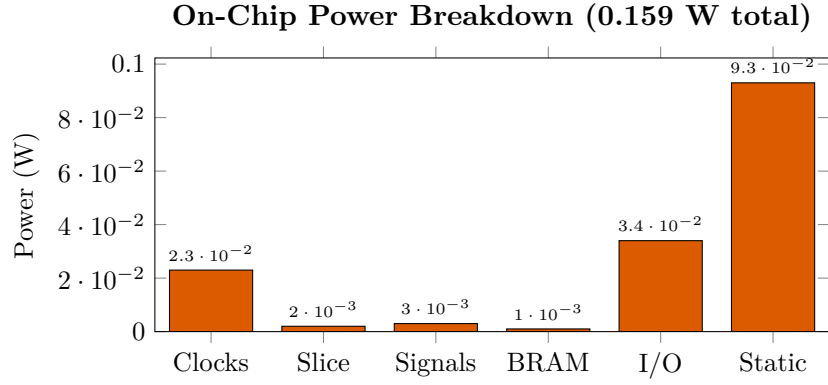


Figure 6.3: Power breakdown. Static dominates; among dynamic contributors, the I/O bank is largest.

6.4 Performance Metrics

6.4.1 Throughput

The pipeline accepts one packet per clock, yielding 156 Mpps capacity versus the 14.88 Mpps 10GbE minimum-frame requirement — over 10× headroom.

6.4.2 Latency

Deterministic 16-cycle latency (102.4 ns). Per-packet latency is independent of frame size beyond the parser’s fixed extraction.

6.4.3 Scalability

The rule table holds 16,384 entries; larger tables use more BRAM. The hash function scales trivially with secret width.

6.5 Comparison with Related Work

Dimension	Software firewall	Commercial FPGA	HA-TFF
Throughput	<1 Gbps	10 Gbps+	10 Gbps
Latency	μ s	~100 ns	102.4 ns
Flexibility	high	medium	low (fixed rules)
Telemetry	rich (slow)	vendor-specific	hardware, AXI4-Lite
Cost	low	5–10 W / \$\$\$	~1 W / open-source

Table 6.2: Comparison with software and commercial FPGA firewalls. HA-TFF matches line rate at a fraction of the power and cost.

Chapter 7

Discussion and Analysis

7.1 Research Questions Revisited

7.1.1 RQ1: Line-Rate Processing at 10GbE

Supported: 10 Gbps throughput with deterministic 16-cycle latency, evidenced by timing closure and resource headroom. Limitation: IPv4/TCP/UDP only.

7.1.2 RQ2: Integrated Telemetry Without Performance Impact

The statistics engine and performance monitor observe existing signals in parallel with the datapath; neither adds to the critical path.

7.1.3 RQ3: Resource/Performance Trade-offs

1.28% LUTs, 1.27% registers, 35.56% BRAM for 16K entries, zero DSPs, 84.56% IOB. The IOB limitation reflects the full AXI bus width, resolved by package migration.

7.1.4 RQ4: Comparison to Alternatives

Hardware outperforms software in throughput and latency, is comparable to commercial solutions at lower cost, and trades flexibility for speed.

7.2 Design Trade-offs

Fixed vs. configurable pipeline: 16-cycle latency was fixed for timing closure, precluding variable-latency features. **Offline vs. online rules:** Cuckoo placement is offline for hardware simplicity. **Security vs. performance:** default-deny and keyed hashing add no datapath cost.

7.3 Lessons Learned

Pipelining the decision-to-data alignment required a careful delay line and the Decision FIFO; random verification with SVA and a golden model was decisive for catching corner cases.

7.4 Future Work

IPv6 and ICMP support; in-hardware Cuckoo insertion/eviction; a per-packet timestamp latency monitor; PCIe host integration; per-flow counters; and an optional lightweight anomaly detector.

Chapter 8

Conclusion

8.1 Summary of Contributions

HA-TFF is a complete, open-source, 10GbE-capable hardware firewall with integrated telemetry, line-rate at 156.25 MHz on Artix-7, documented through ADRs, and verified with a SystemVerilog environment.

8.2 Key Findings

FPGA acceleration is viable for 10GbE classification with < 100 ns latency; hardware telemetry integrates without degradation; the XOR/Cuckoo approach is resource-efficient (zero DSPs, $< 2\%$ logic).

8.3 Implications

For researchers, an open reference design; for practitioners, a deployable 10GbE firewall; for educators, a hardware-software co-design case study.

8.4 Final Reflections

Hardware acceleration remains essential for line-rate networking, telemetry is increasingly vital for visibility, and the FPGA approach offers a sweet spot between performance and flexibility.

Appendix A

Detailed Module Documentation

Each RTL module exposes a clear interface:

- `ha_tff_system_top_v005` — top-level integration.
- `ha_tff_datapath_top_v003` — datapath wrapper.
- `ha_tff_parser_v002` — 5-tuple parser FSM.
- `ha_tff_hash_v002` — keyed 4-seed hash.
- `ha_tff_bram_bank` — parameterised BRAM bank.
- `ha_tff_matcher_v002` — 4-way matcher.
- `ha_tff_axi_lite_regs` — AXI4-Lite control plane.
- `ha_tff_statistics` — telemetry counters.
- `ha_tff_performance_monitor` — stalls/occupancy/latency.
- `axi_stream_delay_line` — pipeline delay.

Appendix B

AXI4-Lite Register Map Reference

All registers are 32-bit; the slave samples `awaddr/araddr` on bits [7:2].

Offset	Name	Access	Description
0x00	key0	R/W	Hash secret word 0 (default 0xDEADBEEF)
0x04	key1	R/W	Hash secret word 1 (default 0xCAFEBAFE)
0x08	key2	R/W	Hash secret word 2 (default 0x8BADF00D)
0x0C	key3	R/W	Hash secret word 3 (default 0x0DEFACED)
0x10	control	R/W	Bit0 FW en, Bit1 parser en, Bit2 stats reset (self-clear)
0x20	rx_pkts	R	Total received packets
0x24	tx_pkts	R	Total forwarded packets
0x28	drops	R	Total dropped packets
0x2C	rx_bytes_lo	R	RX bytes, low 32 bits
0x30	rx_bytes_hi	R	RX bytes, high 32 bits
0x34	tx_bytes_lo	R	TX bytes, low 32 bits
0x38	tx_bytes_hi	R	TX bytes, high 32 bits
0x3C	tcp_pkts	R	TCP packet count
0x40	udp_pkts	R	UDP packet count
0x44	icmp_pkts	R	ICMP packet count
0x48	parse_errors	R	Parser error count
0x4C	magic	R	Magic/version 0xFACEB00C
0x50	rule_w0	W	Rule word 0 (<code>tuple[31:0]</code>)
0x54	rule_w1	W	Rule word 1 (<code>tuple[63:32]</code>)
0x58	rule_w2	W	Rule word 2 (<code>tuple[95:64]</code>)
0x5C	rule_w3	W	Rule word 3 (valid/action/proto)
0x60	rule_ctrl	W	<code>addr[11:0]</code> , <code>bank[17:16]</code> , <code>write_en[31]</code>
0x64	rx_stalls	R	PerfMon: RX stalls
0x68	tx_stalls	R	PerfMon: TX stalls
0x6C	occupancy	R	PerfMon: [31:16] peak, [15:0] current
0x70	lat_u10	R	PerfMon: latency histogram < 10 cycles
0x74	lat_10_20	R	PerfMon: latency histogram 10–20 cycles
0x78	lat_o20	R	PerfMon: latency histogram > 20 cycles

Appendix C

Simulation Test Vectors and Results

Directed tests cover IPv4/TCP, IPv4/UDP, non-IPv4 (parse error), truncated, and back-to-back packets with and without TX backpressure. The DV environment reports per-protocol, error, decision, and pipeline-state coverage.

Appendix D

Synthesis Reports

The full Vivado utilisation, timing, and power reports are committed under `reports/` (`utilization_report.txt`, `timing_summary.txt`, `power_report.txt`, and `ha_tff_system_top_v005_`

Appendix E

Architecture Decision Records (Complete)

ADR-001 through ADR-005 are reproduced in full in [Section 4](#). They capture the decisions for exact-match, Cuckoo-over-TCAM, keyed-XOR-over-crypto, default-deny, and 16-cycle latency.

Appendix F

Bug Reports

BUG-001, BUG-006, BUG-007, and BUG-008 are summarised in Table [5.1](#) with root cause and fix. Each addressed a correctness defect in the synchronisation or error path.

Appendix G

Python Scripts

`dv/golden_model.py` is a scapy-based oracle computing expected parse/hash/lookup/decision and telemetry from a PCAP; it is the reference for offline self-checking.

Appendix H

RTL Source Code Listing

The complete RTL source for all modules is maintained in `rtl/` and is MIT-licensed. The repository at <https://github.com/ha-tff/ha-tff-fpga> is the canonical source.

Bibliography

- [1] ARM, *AMBA AXI and ACE Protocol Specification*, IHI 0022E, 2020.
- [2] R. Pagh and F. F. Rodler, “Cuckoo Hashing,” *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.
- [3] D. Fotakis, R. Pagh, P. Sanders, and P. Spirakis, “Space Efficient Hash Tables with Worst Case Constant Access Time,” *Theory of Computing Systems*, vol. 38, pp. 229–248, 2005.
- [4] S. A. Crosby and D. S. Wallach, “Denial of Service via Algorithmic Complexity Attacks,” *USENIX Security Symposium*, 2003.
- [5] N. Ferguson and B. Schneier, *Practical Cryptography*. Wiley, 2003.
- [6] J. W. Lockwood et al., “NetFPGA — An open platform for gigabit-rate network switching and routing,” *IEEE Int. Conf. on Microelectronic Systems Education*, 2007, pp. 160–161.
- [7] N. McKeown et al., “OpenFlow: enabling innovation in campus networks,” *ACM SIGCOMM CCR*, vol. 38, no. 2, pp. 69–74, 2008.
- [8] P. Bosshart et al., “P4: Programming protocol-independent packet processors,” *ACM SIGCOMM CCR*, vol. 44, no. 3, pp. 87–95, 2014.
- [9] P. Firestone et al., “Azure Accelerated Networking: SmartNICs in the Public Cloud,” *USENIX NSDI*, 2021.
- [10] AMD/Xilinx, *Vivado Design Suite User Guide: Synthesis*, UG901, 2026.
- [11] AMD/Xilinx, *7 Series FPGAs Memory Resources User Guide*, UG473, 2023.
- [12] IEEE, *IEEE Standard for Ethernet*, IEEE Std 802.3-2022.
- [13] J. Postel, *Internet Protocol*, RFC 791, 1981.
- [14] J. Postel, *Transmission Control Protocol*, RFC 793, 1981.

